Day 76: The Week 11 Build — Multi-Head Attention From the Exercise Parts

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 7 | Complete study guide

Watch: A Dive Into Multihead Attention, Self-Attention and Cross-Attention (Machine Learning Studio) | Visual Guide to Transformer Neural Networks, Episode 2: Multi-Head & Self-Attention (Hedu AI)

Script: `day76_multi_head_attention.py` · every number in this guide is copied by program from `day76_run_output.txt`, the real output of that script.

Multi-head attention runs several smaller attention reads side by side and joins them. Today it is assembled from the week's exercise scripts and tested on a task built so that one position must fetch two things at once. With one head the model gets both answers right 0.027 of the time, about what guessing the second answer gives; with two heads, 1.000. A bigger head, four times the training, and attention forced onto the right two pairs all leave one head stuck. The reason is the average: one softmax that looks at two pairs reads their mixture, and the mixture no longer says which value belongs to which key.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

Multi-head attention cuts the queries, keys and values into h equal slices (heads). Each head runs Day 75's scaled dot-product attention on its own slice, so each has its own softmax and can look somewhere different. The heads' outputs are put side by side and mixed by one output matrix W_o. With the total size d fixed, the number of weights does not depend on h.

REAL-WORLD SCENARIO

Asking one assistant to find both the invoice and the delivery note for an order: if they can only hold one folder open, they come back with a blend. Two assistants, one folder each, bring back both.

MATHEMATICS

$$Q = X W_q, \quad K = X W_k, \quad V = X W_v \quad \text{(Day 74)}$$

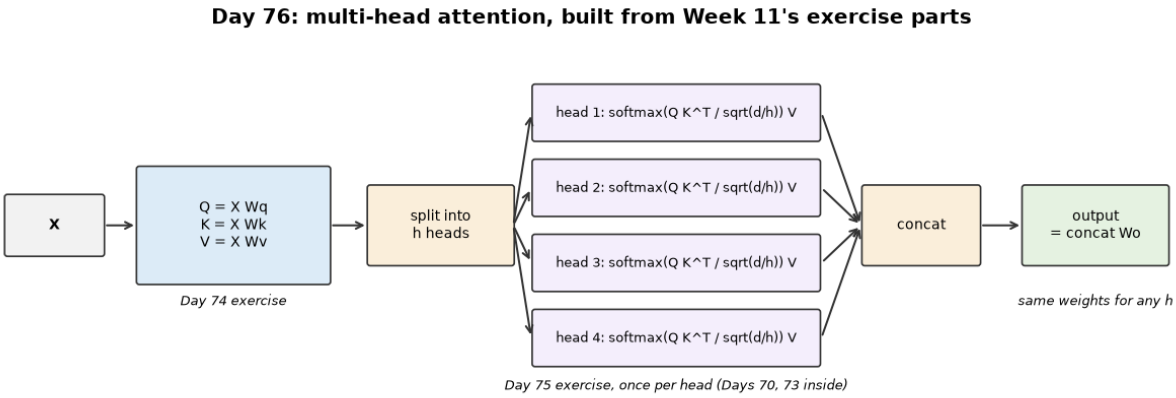
$$head_i = softmax(Q_i K_i^T / \sqrt{(d/h)}) V_i \quad \text{(Day 75, on slice i)}$$

$$output = [head_1 ; ... ; head_h] W_o$$

The parts, and where each one goes

Exercise	Part	Used for
Day 70	masked softmax, weighted read	inside Day 75's function
Day 71	additive read	not used: the Transformer scores with dot products
Day 72	alignment_metrics	what each trained head looks at (Section 6)
Day 73	dot and general reads	the dot score, with learned projections in front
Day 74	Q, K, V from the same sequence	the layer's projections
Day 75	scaled_dot_product_attention	run once per head, verbatim

SYSTEM DIAGRAM



Multi-head attention, with the week's parts named.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The parts fit

DEFINITION — start here, no prior knowledge assumed

The heads are folded into the batch axis, so Day 75's function, written for (examples, positions, size), runs every head in one call. That should equal calling it head by head on column slices.

REAL-WORLD SCENARIO

Sorting four in-trays at once on one big desk, or one tray at a time on four small desks: the same letters end up in the same places.

SPECIFICATION

Data	3 random sequences of 5 positions, size 8, random W_q, W_k, W_v, W_o; h = 1, 2, 4. Then the weight count of the recall model for every h.

Why this test

The reshapes are where a multi-head layer usually goes wrong: a head built from the wrong columns would still run.

IMPLEMENTATION

```
def split_heads(T, h):
    m, n, d = T.shape
    return T.reshape(m, n, h, d // h).transpose(0, 2, 1, 3).reshape(m * h, n, d // h)
def merge_heads(T, h):
    mh, n, dk = T.shape
    return T.reshape(mh // h, h, n, dk).transpose(0, 2, 1, 3).reshape(mh // h, n, h * dk)
def multi_head_attention(X, P, h, mask=None):
    Q, K, V = (split_heads(X @ P[w], h) for w in ("Wq", "Wk", "Wv"))
    Zh, A = scaled_dot_product_attention(Q, K, V, mask)
    Z = merge_heads(Zh, h)
    return Z @ P["Wo"], dict(X=X, Q=Q, K=K, V=V, A=A, Z=Z)
```

ACTUAL OUTPUT

```
 heads  head size   largest |one call - head by head|
     1          8                             0.00e+00
     2          4                             0.00e+00
     4          2                             0.00e+00
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
weights of the recall model (d = 64) for each head count:
  1 head  of size 64: 28736 weights
  2 heads of size 32: 28736 weights
  4 heads of size 16: 28736 weights
  8 heads of size  8: 28736 weights
Actual output from this lesson's run.
```

What it means

The one call and the loop agree exactly (0.00e+00), so head i really is columns i·d/h to (i+1)·d/h – 1. Every head count has 28736 weights: splitting d = 64 into heads changes how the same weights are used, not how many there are.

WHY CHOOSE THIS?

Reusing Day 75's function unchanged keeps one tested piece of code at the centre.

WHY NOT THIS? (TRADEOFFS)

Folding heads into the batch needs every head to have the same size.

2. Gradient check

DEFINITION — start here, no prior knowledge assumed

The whole recall model: embeddings, the multi-head layer and the two answer layers, at one and two heads, and with the attention forced (Section 5).

REAL-WORLD SCENARIO

Checking the wiring of the whole house, not each socket on its own.

MATHEMATICS

dV_i, dQ_i, dK_i *from Day 75's backward on head i; merge them back to full width*

$$dW_q = X^T\, dQ,\quad dW_k = X^T\, dK,\quad dW_v = X^T\, dV,\quad dX = dQ\, W_q^{\,T} + dK\, W_k^{\,T} + dV\, W_v^{\,T}$$

SPECIFICATION

Data	Size 4, 6 possible keys, 4 pairs, 3 examples; every entry nudged by ±1e-5; Day 62's pass rule.

Why this test

Day 75's function was checked on its own; the splitting, merging and W_o are new.

IMPLEMENTATION

```
def multi_head_attention_backward(dO, P, h, c, scores=True):
    flat = lambda T: T.reshape(-1, T.shape[-1])
    g = dict(Wo=flat(c["Z"]).T @ flat(dO))
    dZh = split_heads(dO @ P["Wo"].T, h)
    dQ, dK, dV = scaled_dot_product_attention_backward(dZh, c["Q"], c["K"], c["V"], c["A"])
    if not scores:
        dQ, dK = np.zeros_like(dQ), np.zeros_like(dK)
    dQ, dK, dV = merge_heads(dQ, h), merge_heads(dK, h), merge_heads(dV, h)
    X = c["X"]
    g.update(Wq=flat(X).T @ flat(dQ), Wk=flat(X).T @ flat(dK), Wv=flat(X).T @ flat(dV))
    dX = dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
    return dX, g
```

ACTUAL OUTPUT

```
1 head           : largest |numeric - analytic| = 5.10e-11; every entry passes: True
2 heads          : largest |numeric - analytic| = 7.34e-11; every entry passes: True
2 heads, forced  : largest |numeric - analytic| = 4.08e-11; every entry passes: True
Actual output from this lesson's run.
```

What it means

Every entry passes in all three settings.

WHY CHOOSE THIS?

Every later number is computed with checked gradients.

WHY NOT THIS? (TRADEOFFS)

A tiny model; the trained ones run the same code.

3. Two-key recall

DEFINITION — start here, no prior knowledge assumed

Day 75's recall task, asked about two keys. 16 (key, value) pairs and one query token that names two different keys; the model must give both values, with two separate answer layers.

REAL-WORLD SCENARIO

Asking a shop assistant for the price of two items in one question.

SPECIFICATION

Models	d = 64 split into 1, 2, 4 or 8 heads; the query token is E_a[first key] + E_b[second key], so the two are told apart.
Training	Day 75's: plain SGD lr 0.1, clip 1.0, batches of 64, 2,000 steps, three seeds; 2,000 fixed test examples.

Why this test

One position must collect two separate things. That is exactly what one softmax cannot do without blending.

IMPLEMENTATION

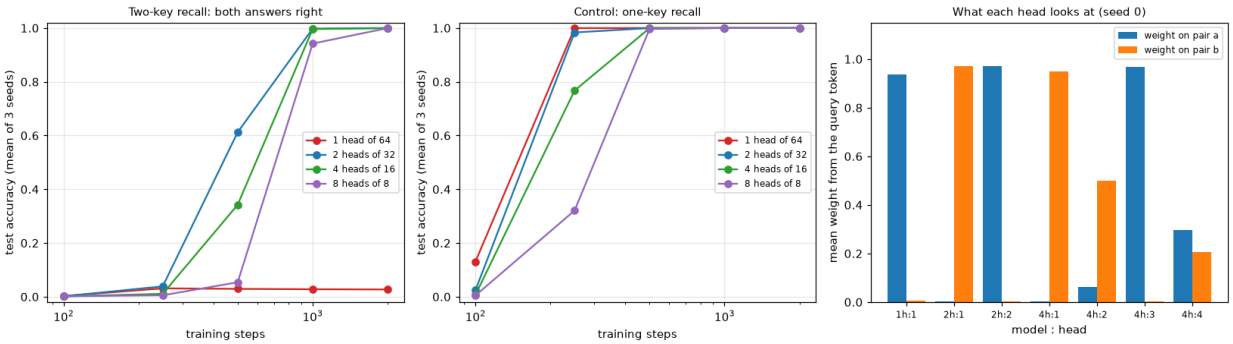
```
def forward(self, keys, vals, qa, qb, force=None):
    P = self.P
    pairs = P["Ek"][keys] + P["Ev"][vals]
    query = P["Ea"][qa] + P["Eb"][qb]
    X = np.concatenate([pairs, query[:, None, :]], axis=1)
    out, c = multi_head_attention(X, P, self.h)
    if force is not None:
        c = self.forced_read(c, force)
        out = c["Z"] @ P["Wo"]
    o = out[:, -1]
    pa, pb = softmax(o @ P["Ua"] + P["ba"]), softmax(o @ P["Ub"] + P["bb"])
    self.cache = dict(c=c, keys=keys, vals=vals, qa=qa, qb=qb, o=o, pa=pa, pb=pb, forced=force is
        not None)
    return pa, pb
```

ACTUAL OUTPUT

```
share with BOTH answers right, mean of three seeds:
 heads  size    @100    @250    @500   @1000   @2000   first  second
     1    64   0.003   0.031   0.029   0.028   0.027   0.349   0.677
     2    32   0.002   0.039   0.612   0.998   1.000   1.000   1.000
     4    16   0.001   0.011   0.342   0.996   0.999   1.000   0.999
     8     8   0.002   0.005   0.054   0.942   1.000   1.000   1.000
('first' and 'second': each answer on its own, at the end)
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
per seed, both right at the end:
  1 head : 0.033  0.026  0.022
  2 heads: 1.000  1.000  1.000
  4 heads: 1.000  1.000  0.999
  8 heads: 1.000  1.000  1.000
Actual output from this lesson's run.
```



Left: two-key recall, both answers right. Middle: the one-key control (Section 4). Right: where each head of the seed-0 models looks (Section 6).

What it means

One head never learns it: both answers right 0.027 of the time at the end, and no seed above 0.033. It learns one of the two answers; which one depends on the seed (first 0.349, second 0.677 on average over the seeds).

Two or more heads solve it (1.000 for 2 and 8 heads, 0.999 for 4). More, smaller heads start later: at 500 steps 2 heads are at 0.612, 4 at 0.342 and 8 at 0.054.

WHY CHOOSE THIS?

Heads let one position gather several things at once, at no cost in weights.

WHY NOT THIS? (TRADEOFFS)

A toy task built to need exactly two lookups; real tasks need some unknown number.

4. The control: one-key recall

DEFINITION — start here, no prior knowledge assumed

The same models, asked about the same key twice: one pair to fetch.

REAL-WORLD SCENARIO

Asking for the price of one item: one assistant is enough.

Why this test

If one head also failed here, Section 3 would show a broken model, not a limit of one softmax.

ACTUAL OUTPUT

```
 heads  size    @100    @250    @500   @1000   @2000
     1    64   0.131   0.999   1.000   1.000   1.000
     2    32   0.024   0.983   1.000   1.000   1.000
     4    16   0.007   0.767   0.999   1.000   1.000
     8     8   0.005   0.321   0.997   1.000   1.000
Actual output from this lesson's run.
```

What it means

Every head count reaches 1.000, and one head is the fastest (0.999 at 250 steps, against 0.321 for 8 heads of size 8). One head is not weak in general; it fails only when one position needs two things. With nothing to split, smaller heads are a cost; a likely reason (not tested here) is that each head's scores come from only d/h numbers.

WHY CHOOSE THIS?

The control separates the two-lookup limit from any general weakness.

WHY NOT THIS? (TRADEOFFS)

One task, one size.

5. Why one head fails: size, time, or the average?

DEFINITION — start here, no prior knowledge assumed

Three follow-ups on two-key recall. (a) One head twice as large. (b) One head trained four times as long. (c) The attention FORCED onto the two right pairs: one head puts half its weight on each; with two heads, each puts all its weight on one pair. Then the queries and keys get no gradient; the rest is trained.

REAL-WORLD SCENARIO

Hiring a bigger assistant, giving them more time, or handing them the two right folders already mixed in one pile.

MATHEMATICS

one head, forced: read = ½ (v_a + v_b), $v_j = (E_k[key_j] + E_v[value_j])\, W_v$

the sum is the same whichever value goes with which key

Why this test

Forcing the weights removes the question of whether one head can *find* both pairs; what is left is whether it can *answer* from what it reads.

IMPLEMENTATION

```
def forced_read(self, c, force):
    A = c["A"].copy()
    m = len(force[0]); r = np.arange(m)
    A = A.reshape(m, self.h, A.shape[1], A.shape[2])
    A[:, :, -1, :] = 0.0
    if self.h == 1:
        A[r, 0, -1, force[0]] += 0.5; A[r, 0, -1, force[1]] += 0.5
    else:
        A[r, 0, -1, force[0]] = 1.0
        A[r[:, None], np.arange(1, self.h)[None, :], -1, force[1][:, None]] = 1.0
    A = A.reshape(m * self.h, A.shape[2], A.shape[3])
    Zh = np.einsum("mij,mjd->mid", A, c["V"])
    return dict(c, A=A, Z=merge_heads(Zh, self.h))
```

ACTUAL OUTPUT

```
model                          first  second    both   both, per seed
one head, d = 128              0.405   0.631   0.082   0.086 0.080 0.081
one head, 8000 steps           0.433   0.601   0.054   0.046 0.041 0.074
one head, forced 1/2 + 1/2     0.518   0.512   0.328   0.331 0.340 0.314
two heads, forced              1.000   1.000   1.000   1.000 1.000 1.000
Actual output from this lesson's run.
```

What it means

Size and time do not fix it: one head of 128 reaches 0.082 with both answers right, and 8,000 steps 0.054.

The average is the reason. Even when handed the right two pairs, one head gets each answer right only about half the time (0.518 and 0.512): it can tell the two values apart from everything else, but not which belongs to which key, so it picks one of the two. Two forced heads give 1.000. Both right comes out at 0.328, more than the 0.25 that two independent coin flips would give; that excess is not investigated here.

WHY CHOOSE THIS?

Separates finding the information from using it.

WHY NOT THIS? (TRADEOFFS)

The forced models are trained without learning where to look; three seeds.

6. What each head looks at

DEFINITION — start here, no prior knowledge assumed

Day 72's measurements on each head's weights from the query token, against pair a and against pair b.

REAL-WORLD SCENARIO

Watching which folder each assistant opens.

Why this test

Section 3 shows that the heads solve the task; this shows how they divide it.

IMPLEMENTATION

```
def alignment_metrics(W, target):
    j = np.arange(len(target))
    entropy = -(W * np.log(W + 1e-12)).sum(axis=1)
    return (W.argmax(axis=1) == target).mean(), W[j, target].mean(), entropy.mean()
```

ACTUAL OUTPUT

```
model      head  argmax on a  weight on a  argmax on b  weight on b  entropy
1 head        1        1.000        0.937        0.000        0.005    0.328
2 heads       1        0.000        0.002        1.000        0.971    0.154
2 heads       2        1.000        0.971        0.000        0.001    0.156
4 heads       1        0.000        0.003        1.000        0.949    0.244
4 heads       2        0.068        0.064        0.629        0.500    1.351
4 heads       3        1.000        0.967        0.000        0.002    0.170
4 heads       4        0.440        0.296        0.312        0.206    1.755
(entropy over 17 tokens: log 17 = 2.83 means spread evenly)
Actual output from this lesson's run.
```

What it means

The single head looks at one pair only (0.937 on pair a, 0.005 on b). With two heads, each takes one pair (0.971 and 0.971). With four, two heads do the work (0.949 on b, 0.967 on a); the other two are more spread out (entropy 1.351 and 1.755), though head 2 still puts 0.500 on pair b. Nothing in training asks every head to have its own job.

WHY CHOOSE THIS?

The heads' split is visible and measurable.

WHY NOT THIS? (TRADEOFFS)

Seed 0 only; another seed may divide the work differently.

7. What more heads cost

DEFINITION — start here, no prior knowledge assumed

One training step's time for each head count, in this run. The weight count is the same for all.

REAL-WORLD SCENARIO

Four small desks take more walking between them than one big desk, even with the same paper.

ACTUAL OUTPUT

```
 heads  ms per step  vs one head
     1         7.28         1.00
     2         7.69         1.06
     4        10.01         1.37
     8        16.17         2.22
Actual output from this lesson's run.
```

What it means

Two heads cost 1.06 times one head here, eight heads 2.22 times: the same arithmetic in more, smaller pieces. Only ratios within this run are meaningful (this machine's speed varies between runs).

WHY CHOOSE THIS?

No extra weights.

WHY NOT THIS? (TRADEOFFS)

More, smaller matrix products are slower in NumPy, and learn later (Section 4).

Interview preparation — technical questions

Q: Write multi-head attention.

A: head_i = softmax(Q_i K_i^T / sqrt(d/h)) V_i on column slices i; output = [head_1; ...; head_h] Wo.

Q: How many weights does it add over one head of the same size?

A: None: Wq, Wk, Wv and Wo are d x d whatever h is (28736 here for every h).

Q: Why scale by sqrt(d/h) and not sqrt(d)?

A: Each head's scores are dot products of d/h numbers (Day 75).

Q: How do you run all heads with one function call?

A: Reshape (m, n, d) to (m*h, n, d/h): each head becomes a batch entry.

Q: What did one head score on two-key recall?

A: 0.027 both right; two heads 1.000.

Interview preparation — theoretical questions

Q: Why can one head not fetch two things?

A: One softmax over both reads their average, which loses which value goes with which key (Section 5: 0.518 and 0.512 even with the right pairs given).

Q: Are more heads always better?

A: No: on one-key recall 8 heads of 8 learned later than one head of 64, and in the 4-head model two heads were more spread out than the other two.

Q: What does Wo do?

A: Mixes the heads' outputs, so the next layer sees one d-sized vector.

Q: How would you check what heads learned?

A: Measure each head's weights against known targets (Day 72's metrics).

Key takeaway and what's next

KEY TAKEAWAY

Multi-head attention is Day 75's function run on h slices, joined by W_o, with no extra weights. It matters when one position must collect several things: one softmax that looks at two places reads a blend, and the blend loses which value belongs to which key. With one thing to fetch, one head is enough, and smaller heads only learn later.

- One call on all heads equals Day 75's function head by head: 0.00e+00.
- 28736 weights for 1, 2, 4 and 8 heads.
- Two-key recall, both right: 1 head 0.027; 2 heads 1.000; 4 heads 0.999; 8 heads 1.000.
- One head of 128: 0.082; 8,000 steps: 0.054; forced onto the right pairs: 0.518 and 0.512 per answer.
- One-key recall: all reach 1.000; 8 heads at 250 steps 0.321 against 0.999 for one head.

The week in one table

Day	What it built	What it showed
70	the bottleneck	one state holds 4 digits perfectly, 20 not
71	additive attention	1.000 at every length where Day 70 failed
72	alignment heatmaps	the diagonal, and where it breaks past training lengths
73	dot and general scores	all work at hidden 32; the dot score fails at 16
74	self-attention	order-blind, proved on real sentences
75	the 1/√d scaling	its benefit is the starting point
76	multi-head attention	one head cannot fetch two things

Suggested watch order, if starting from zero

- 1. Visual Guide to Transformer Neural Networks, Episode 2: Multi-Head & Self-Attention (Hedu AI) — heads drawn step by step.
- 2. A Dive Into Multihead Attention, Self-Attention and Cross-Attention (Machine Learning Studio) — multi-head, self- and cross-attention side by side.

Day 77 preview

- **Week 12, the Transformer, starts with positional encoding.** Self-attention still cannot tell one word order from another (Day 74); Day 77 adds a vector per position and tests it on Week 9's tagging task.
- Day 76 is the week's build, so there is no exercise script today; Week 12's exercises start on Day 77.